

4 — Linear models

Machine Learning (MPA-MLR)

Tomáš Vičar

Department of Biomedical Engineering · FEEC, Brno University of Technology

2025

1 · One equation, two tasks

One dot product of weights and features predicts a number or a class — everything else is the choice of loss.

Linear models: regression vs. classification

Our general linear equation $\mathbf{z} = \mathbf{W}^\top \mathbf{x}$ (last lecture) if we want to predict just one value:

$$g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d$$

Bias trick (augmenting features): we can write an **affine** function as a purely linear one by extending $\mathbf{x}$ (we will use the syntax $w/$ or w/o b interchangeably):

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b, \quad \tilde{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}, \quad \tilde{\mathbf{w}} = \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix} \Rightarrow f(\mathbf{x}) = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$$

- **Linear regression:** continuous prediction

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} \in \mathbb{R}$$

- **Linear classification (2 classes):** class prediction

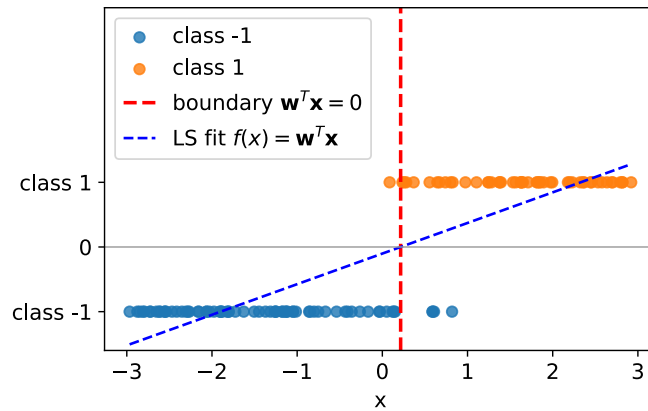
$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b) = \text{sign}(\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}) \in \{-1, +1\}$$

Decision boundary: $\mathbf{w}^\top \mathbf{x} + b = 0$

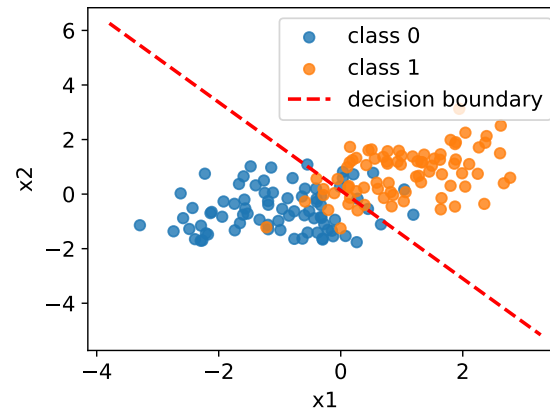
Linear classification: visualizations

4

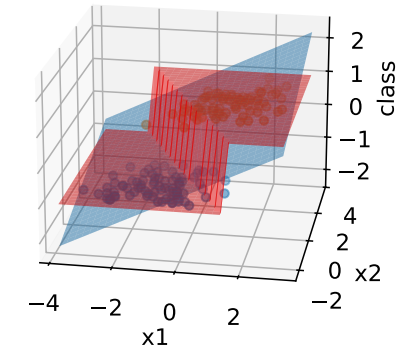
1D classification



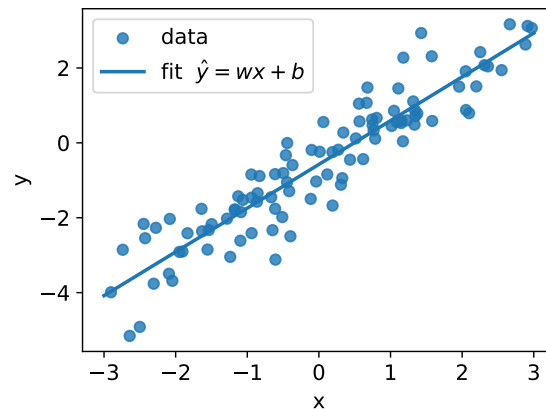
2D classification



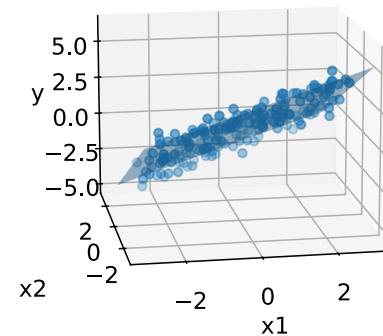
2D classification



1D regression



2D regression



Fit a linear model

Data and model:

$$\{(\mathbf{x}_i, y_i)\}_{i=1}^n, \quad \mathbf{x}_i \in \mathbb{R}^d, \quad y_i \in \mathbb{R} \text{ (regression)} \text{ or } \{-1, +1\} \text{ (classification)}$$

Prediction is $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ or $\text{sign}(\mathbf{w}^\top \mathbf{x})$ — we just need to find $\mathbf{w}$:

Optimization (training): choose $\mathbf{w}$ to minimize average loss + regularization

$$\min_{\mathbf{w}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, \mathbf{w}^\top \mathbf{x}_i) + \lambda \Omega(\mathbf{w})$$

Typical regularizers: $\Omega(\mathbf{w}) = \|\mathbf{w}\|_2^2$ (Ridge — L_2), $\Omega(\mathbf{w}) = \|\mathbf{w}\|_1$ (LASSO — L_1).

Loss choices:

- **Regression:** MSE, MAE
- **Classification:** SVM, Perceptron, LDA, Logistic (log-loss)

Loss choices for linear models

6

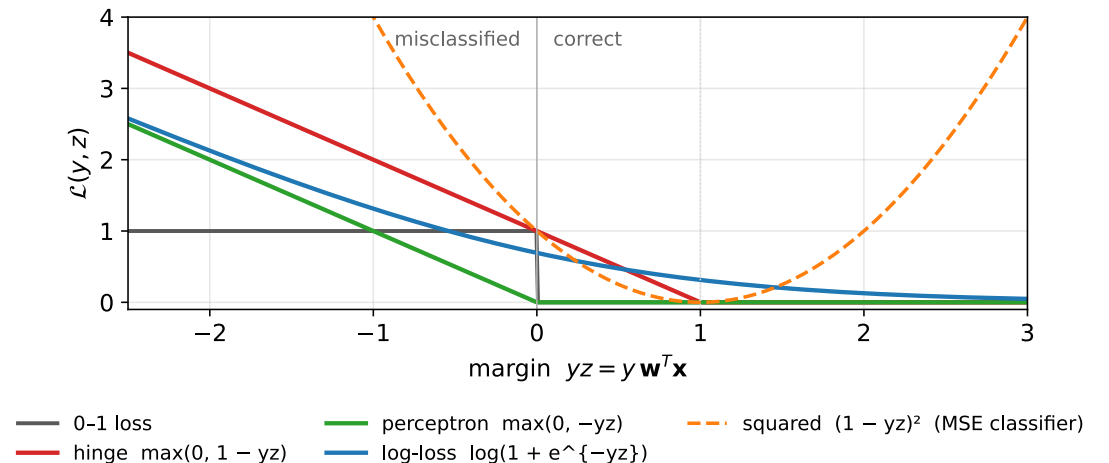
Let $z = \mathbf{w}^\top \mathbf{x}$.

Regression

- **MSE (squared error):** $\mathcal{L}_{\text{MSE}}(y, z) = (y - z)^2$ — smooth, closed form, sensitive to outliers.
- **MAE (absolute error):** $\mathcal{L}_{\text{MAE}}(y, z) = |y - z|$ — robust to outliers; solved via LP/IRLS (no closed form).

Classification (2 classes)

- **SVM** — max-margin with *hinge* loss
 $\mathcal{L}_{\text{hinge}}(y, z) = \max(0, 1 - yz)$
- **Perceptron** — perceptron loss $\mathcal{L}_{\text{perc}}(y, z) = \max(0, -yz)$
- **LDA** — Fisher criterion
- **Logistic (log-loss)** $\mathcal{L}_{\text{log}}(y, z) = \log(1 + e^{-yz})$



2 · Linear regression

From a line through two points to ordinary least squares, robust fits and regularization.

Linear regression with two points

Model (1D feature + intercept): $y = w x + b$

Equations to satisfy:

$$y_1 = w x_{11} + b$$

$$y_2 = w x_{21} + b$$

Matrix form:

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & x_{11} \\ 1 & x_{21} \end{bmatrix}}_X \underbrace{\begin{bmatrix} b \\ w \end{bmatrix}}_{\tilde{\mathbf{w}}} \iff \mathbf{y} = X \tilde{\mathbf{w}}.$$

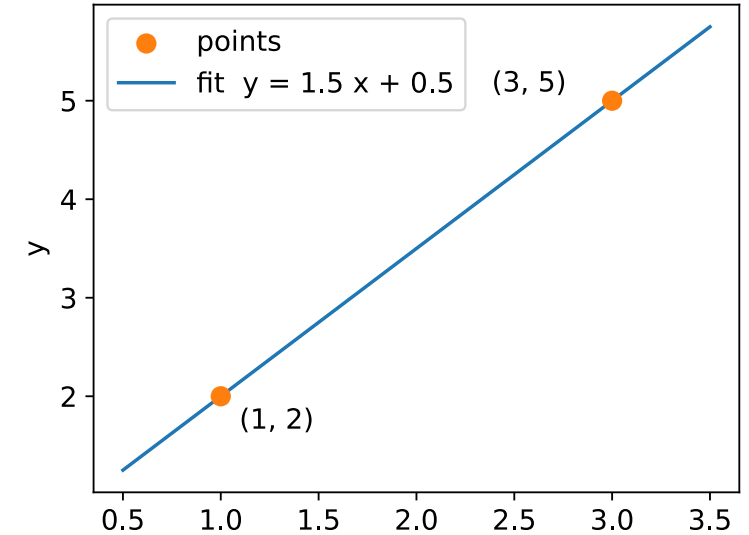
Inverse-form solution: $\tilde{\mathbf{w}} = X^{-1} \mathbf{y}$ · by hand for $(1, 2), (3, 5)$: $X = \begin{bmatrix} 1 & 1 \\ 1 & 3 \end{bmatrix}$, $\det X = 2$, $X^{-1} = \frac{1}{2} \begin{bmatrix} 3 & -1 \\ -1 & 1 \end{bmatrix}$,

$$\tilde{\mathbf{w}} = \frac{1}{2} \begin{bmatrix} 3 \cdot 2 - 5 \\ -2 + 5 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 1.5 \end{bmatrix}$$

Prediction for a new input x_{new} and for more new samples in the feature matrix X_{new} :

$$\hat{y}_{\text{new}} = w x_{\text{new}} + b, \quad \hat{\mathbf{y}}_{\text{new}} = X_{\text{new}} \tilde{\mathbf{w}}$$

For two points there exists a straight line that goes through both points. For more points it is not possible $\rightarrow$ we need to make $\mathbf{y}$ and $X \tilde{\mathbf{w}}$ as similar as possible.

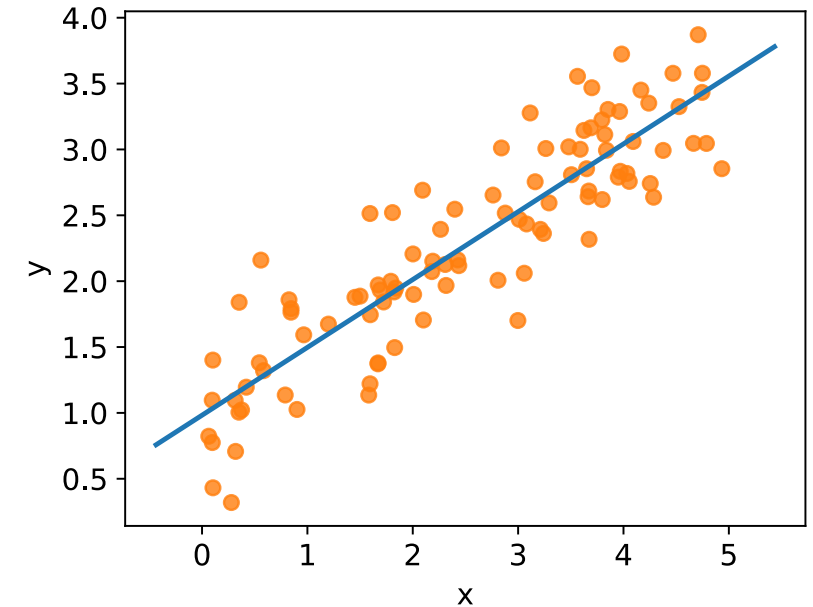


MSE linear regression (OLS)

Ordinary least squares. We want predictions $X\mathbf{w}$ to be as close as possible to $\mathbf{y}$. We can define it with MSE (L_2 norm).

$$\underset{\mathbf{w}}{\text{minimize}} \quad \sum_{i=1}^n (\mathbf{x}_i^\top \mathbf{w} - y_i)^2 = \|X\mathbf{w} - \mathbf{y}\|_2^2$$

This is a simple convex optimization without constraints. We can employ various approaches (e.g. gradient descent), but a closed-form solution is also available.



Objective (MSE): $g(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|_2^2 = (X\mathbf{w} - \mathbf{y})^\top (X\mathbf{w} - \mathbf{y})$

Derivative of cost: $\nabla g(\mathbf{w}) = 2X^\top (X\mathbf{w} - \mathbf{y})$ · set gradient to zero: $2X^\top (X\mathbf{w} - \mathbf{y}) = 0$

Closed-form solution: $\boxed{\mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{y} = X^\dagger \mathbf{y}}$

Here $X^\dagger = (X^\top X)^{-1} X^\top$ denotes the Moore–Penrose **pseudoinverse**.

OLS by hand: three points

Data: $(0, 1), (1, 2), (2, 2)$ — no line passes through all three.

$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}, \quad X^\top X = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}, \quad X^\top \mathbf{y} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

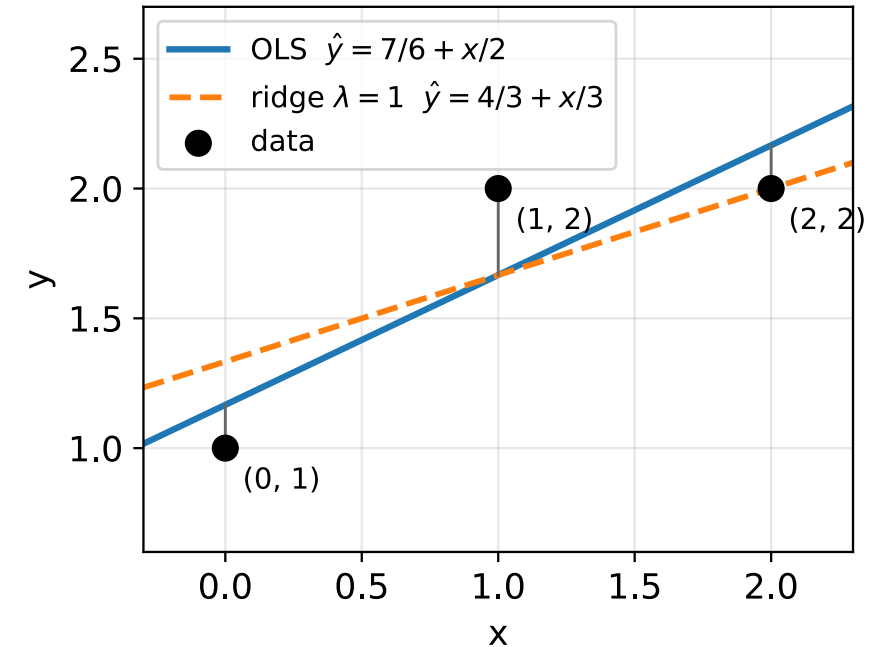
$$\tilde{\mathbf{w}} = (X^\top X)^{-1} X^\top \mathbf{y} = \frac{1}{6} \begin{bmatrix} 5 & -3 \\ -3 & 3 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 7/6 \\ 1/2 \end{bmatrix} \Rightarrow \hat{y} = \frac{7}{6} + \frac{1}{2}x$$

Residuals $(-\frac{1}{6}, \frac{1}{3}, -\frac{1}{6})$, sum of squares $\frac{1}{6}$ — the smallest any line can achieve.

Ridge with $\lambda = 1$ on the slope only (the intercept is usually not penalized):

$$\tilde{\mathbf{w}} = \left(X^\top X + \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \right)^{-1} X^\top \mathbf{y} = \frac{1}{9} \begin{bmatrix} 6 & -3 \\ -3 & 3 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 4/3 \\ 1/3 \end{bmatrix}$$

The slope shrinks from $\frac{1}{2}$ to $\frac{1}{3}$; the intercept moves to keep the line near the data.



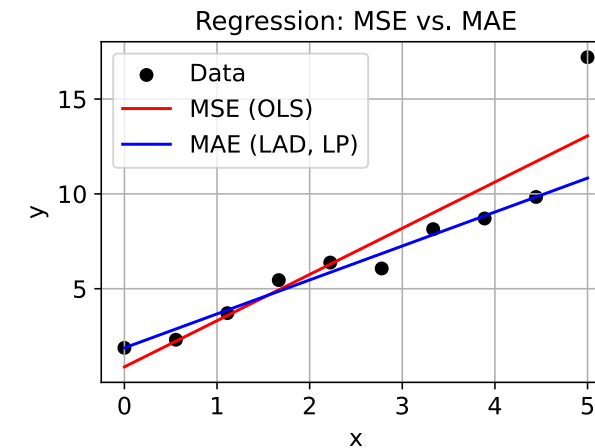
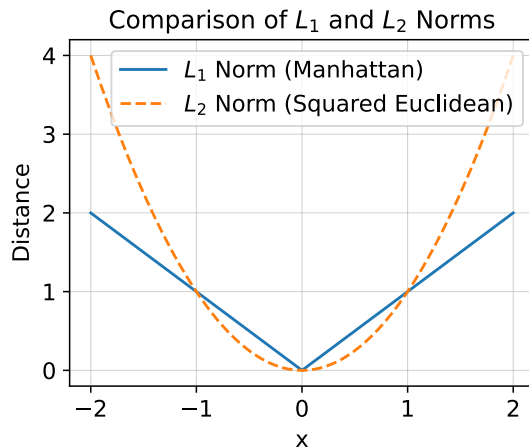
Linear regression with MAE (LAD)

Least absolute deviations. Objective (MAE):

$$\underset{\mathbf{w}}{\text{minimize}} \quad \sum_{i=1}^n |\mathbf{x}_i^\top \mathbf{w} - y_i| = \|X\mathbf{w} - \mathbf{y}\|_1$$

No closed-form solution. Solved via optimization methods (e.g. LP).

$$\|\mathbf{x}\|_2 = \|\mathbf{x}\| = \sqrt{\sum_i x_i^2} \quad \cdot \quad \|\mathbf{x}\|_2^2 = \mathbf{x}^\top \mathbf{x} = \sum_i x_i^2 \quad \cdot \quad \|\mathbf{x}\|_1 = \sum_i |x_i|$$



L_1 is less sensitive to large values (outliers) and more sensitive to very small values (very small values are still penalized) $\rightarrow$ LAD is robust to outliers.

“Small values squared are very small and thus ignored — e.g. $0.01^2 = 0.0001$.”

Ridge vs. LASSO regularization for linear regression

12

Ridge regression (L_2)

$$\underset{\mathbf{w}}{\text{minimize}} \quad \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_2^2$$

- Shrinks coefficients smoothly.
- Keeps all features (no exact zeros).
- Closed form: $\mathbf{w} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$.

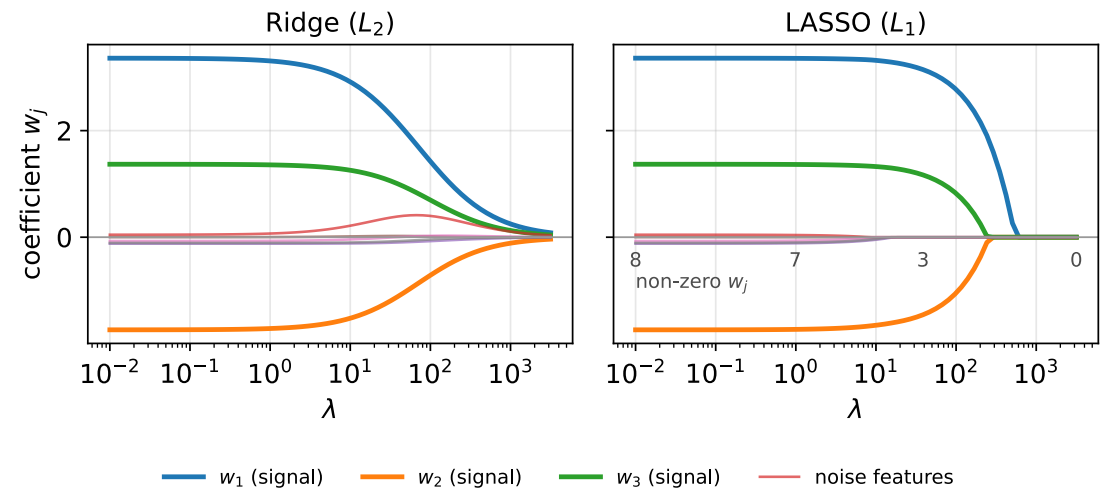
λ is a hyper-parameter adjusting the amount of regularization (weighing between the fit and the regularization term). For LASSO: $\uparrow \lambda \rightarrow$ fewer features selected.

The same regularization can be used for other methods (LAD, linear classifiers) with similar properties.

LASSO regression (L_1)

$$\underset{\mathbf{w}}{\text{minimize}} \quad \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_1$$

- Performs feature selection (some $w_j = 0$).
- Robust to irrelevant features.
- No closed form, solved by convex optimization.



3 · Nonlinear features and kernels

The model stays linear in the parameters — we only change what we feed into it.

Polynomial regression

14

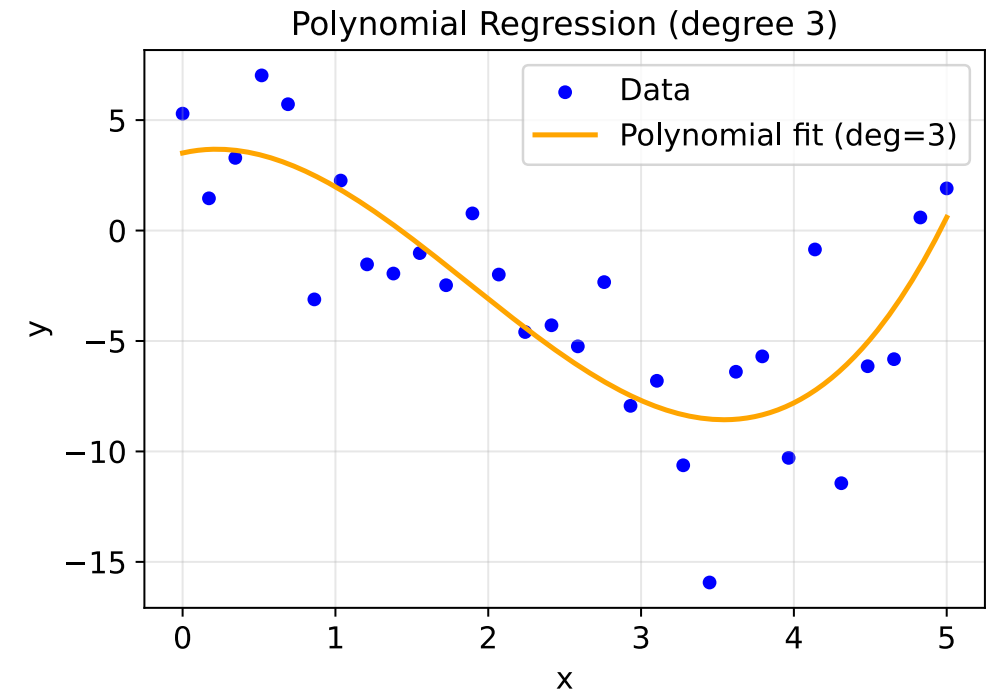
We can use the same approach as for linear regression to fit a non-linear function.

Example: m -th degree polynomial of 1D data

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_m x_i^m$$

Which is equivalent to transforming features and then using linear regression:

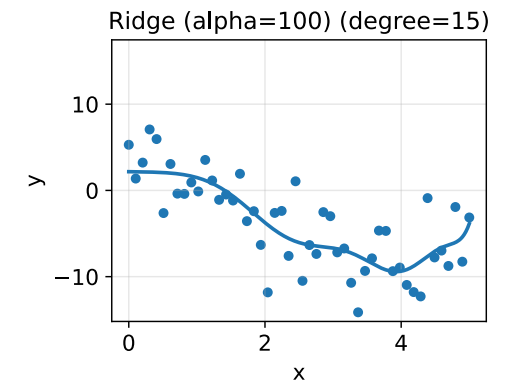
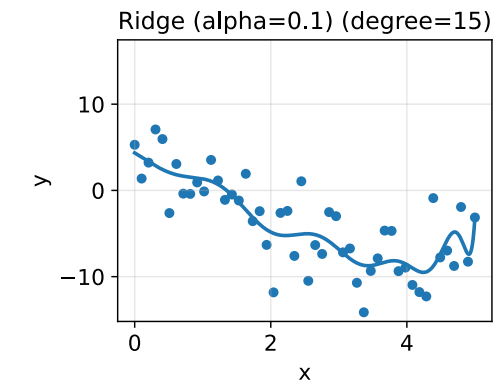
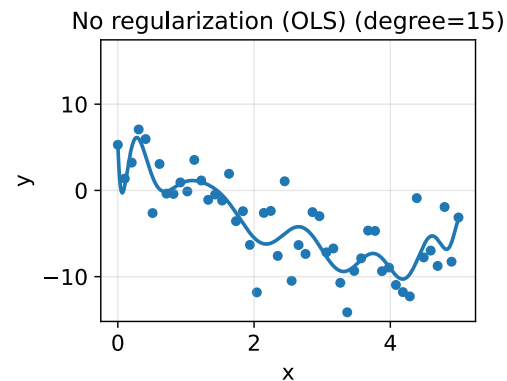
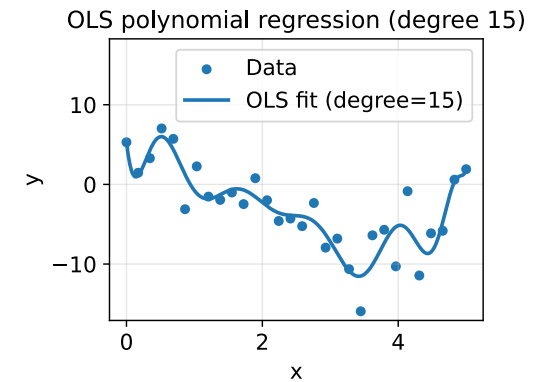
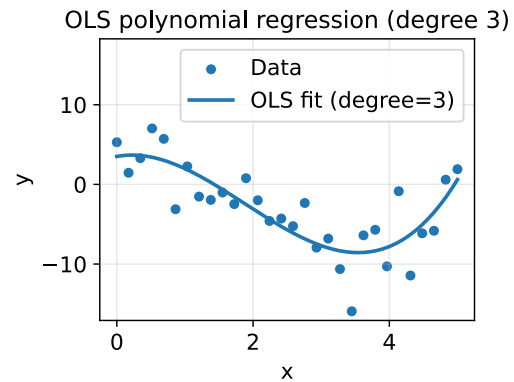
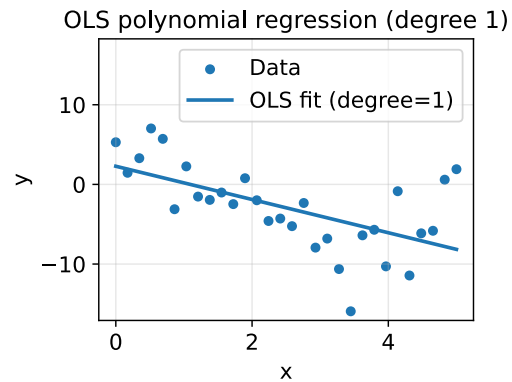
$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^m \\ 1 & x_2 & x_2^2 & \cdots & x_2^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^m \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_m \end{bmatrix}$$



Estimation of coefficients is the same as in linear regression — only X is transformed. And similarly to PCA, we can also do an implicit expansion using kernels (but we need the dual form). [code example](#)

Effect of polynomial degree and regularization

15



We have 2 hyperparameters — the degree of the polynomial and the amount of regularization (λ).

Which one is an example of over-fitting and which one of under-fitting?

Polynomial transform — degree 3 in 2D (dimensionality quickly grows with input dimension and degree)

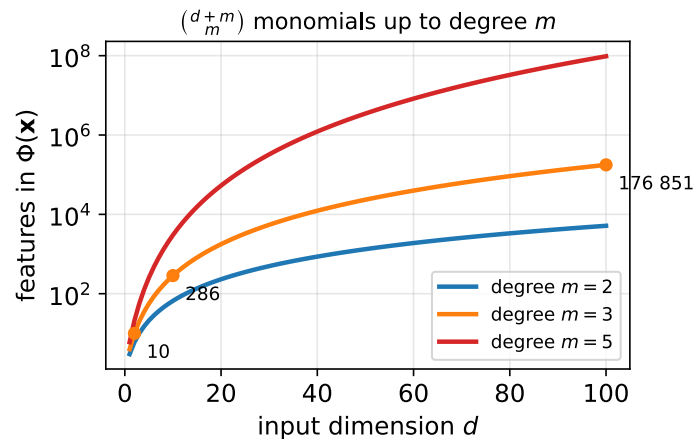
$$\Phi^{(3)}(\mathbf{x}) = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3]^\top$$

Problem: explicit mapping $\Phi(\mathbf{x})$ can lead to very high (even infinite) dimensional feature spaces. Computing $\Phi(\mathbf{x})$ directly may be expensive or infeasible.

Key idea: many ML algorithms have a *dual form*, where they depend *only on inner products* $\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j)$:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j)$$

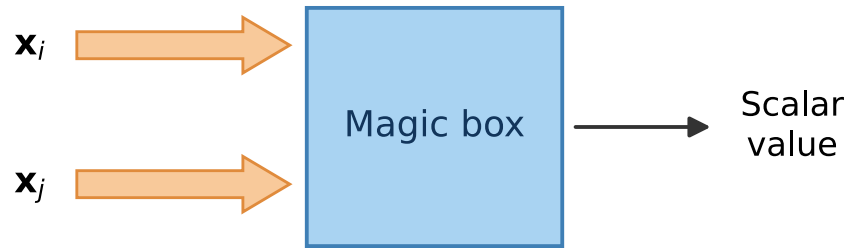
If we can compute $k(\mathbf{x}_i, \mathbf{x}_j)$ **without computing $\Phi(\mathbf{x})$ explicitly**, we get the benefits of high-dimensional features at low cost.



Monomials up to degree m in d variables: $\binom{d+m}{m}$

- $d = 2, m = 3: \binom{5}{3} = 10$ (the vector above)
- $d = 10, m = 3: 286$
- $d = 100, m = 3: 176\,851$
- RBF kernel: infinite-dimensional Φ

Kernel trick: a worked example



Two equivalent views:

1. Explicit feature expansion:

$$\Phi(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2} x_1 x_2)$$

$$\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j) = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2x_{i1} x_{i2} x_{j1} x_{j2}$$

2. Direct computation (kernel trick):

$$(\mathbf{x}_i^\top \mathbf{x}_j)^2 = (x_{i1} x_{j1} + x_{i2} x_{j2})^2 = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2x_{i1} x_{i2} x_{j1} x_{j2}$$

Check by hand: $\mathbf{x}_i = (1, 2), \mathbf{x}_j = (3, 1)$

- $\Phi(\mathbf{x}_i) = (1, 4, 2\sqrt{2}), \Phi(\mathbf{x}_j) = (9, 1, 3\sqrt{2})$
- $\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j) = 9 + 4 + 12 = 25$
- $(\mathbf{x}_i^\top \mathbf{x}_j)^2 = (3 + 2)^2 = 25 \checkmark$

Conclusion: computing $\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j)$ explicitly is the same as computing $(\mathbf{x}_i^\top \mathbf{x}_j)^2$. $\Rightarrow$ the kernel function avoids the explicit high-dimensional mapping.

Kernel function: a function $k : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that

$$k(\mathbf{x}_i, \mathbf{x}_j) = \langle \Phi(\mathbf{x}_i), \Phi(\mathbf{x}_j) \rangle = \Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j)$$

for some (possibly high-dimensional) feature map $\Phi(\cdot)$.

Examples:

- Linear kernel: $k(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$
- Polynomial kernel: $k(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^\top \mathbf{x}_j + c)^p$
- RBF (Gaussian) kernel: $k(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$

Kernel matrix (Gram matrix): for n samples $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$,

$$K = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & k(\mathbf{x}_1, \mathbf{x}_2) & \dots & k(\mathbf{x}_1, \mathbf{x}_n) \\ k(\mathbf{x}_2, \mathbf{x}_1) & k(\mathbf{x}_2, \mathbf{x}_2) & \dots & k(\mathbf{x}_2, \mathbf{x}_n) \\ \vdots & \vdots & \ddots & \vdots \\ k(\mathbf{x}_n, \mathbf{x}_1) & k(\mathbf{x}_n, \mathbf{x}_2) & \dots & k(\mathbf{x}_n, \mathbf{x}_n) \end{bmatrix}$$

The kernel function can “replace” $\mathbf{x}_i^\top \mathbf{x}_j$ and the kernel matrix can “replace” $XX^\top$.

Primal form

$$\underset{\mathbf{w}}{\text{minimize}} \quad \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_2^2$$

closed-form solution:

$$\mathbf{w}^* = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$$

prediction of a new sample:

$$\hat{y}_{\text{new}} = \mathbf{x}_{\text{new}}^\top \mathbf{w}$$

prediction for *many* new samples (feature matrix $X_{\text{new}} \in \mathbb{R}^{m \times d}$):

$$\hat{\mathbf{y}}_{\text{new}} = X_{\text{new}} \mathbf{w}$$

Dual form

$$\underset{\boldsymbol{\alpha}}{\text{minimize}} \quad \frac{1}{2} \boldsymbol{\alpha}^\top (XX^\top + \lambda I) \boldsymbol{\alpha} - \mathbf{y}^\top \boldsymbol{\alpha}$$

closed-form solution:

$$\boldsymbol{\alpha}^* = (XX^\top + \lambda I)^{-1} \mathbf{y}$$

$$\mathbf{w} = \sum_{i=1}^n \alpha_i \mathbf{x}_i = X^\top \boldsymbol{\alpha}$$

prediction of a new sample:

$$\hat{y}_{\text{new}} = \sum_{i=1}^n \alpha_i \mathbf{x}_i^\top \mathbf{x}_{\text{new}} = \boldsymbol{\alpha}^\top X \mathbf{x}_{\text{new}}$$

prediction for *many* new samples:

$$\hat{\mathbf{y}}_{\text{new}} = X_{\text{new}} X^\top \boldsymbol{\alpha}$$

Again, the primal form uses the $d \times d$ matrix $X^\top X$ and the dual form uses the $n \times n$ matrix $XX^\top$. Which one is faster to use depends on whether you have more features or more samples.

Dual form

$$\underset{\boldsymbol{\alpha}}{\text{minimize}} \quad \frac{1}{2} \boldsymbol{\alpha}^\top (X X^\top + \lambda I) \boldsymbol{\alpha} - \mathbf{y}^\top \boldsymbol{\alpha}$$

closed-form solution:

$$\boldsymbol{\alpha}^* = (X X^\top + \lambda I)^{-1} \mathbf{y}$$

prediction of a new sample:

$$\hat{y}_{\text{new}} = \sum_{i=1}^n \alpha_i \mathbf{x}_i^\top \mathbf{x}_{\text{new}}$$

prediction for *many* new samples:

$$\hat{\mathbf{y}}_{\text{new}} = X_{\text{new}} X^\top \boldsymbol{\alpha}$$

Kernel ridge regression

$$\underset{\boldsymbol{\alpha}}{\text{minimize}} \quad \frac{1}{2} \boldsymbol{\alpha}^\top (K + \lambda I) \boldsymbol{\alpha} - \mathbf{y}^\top \boldsymbol{\alpha}$$

closed-form solution:

$$\boldsymbol{\alpha}^* = (K + \lambda I)^{-1} \mathbf{y}$$

prediction of a new sample:

$$\hat{y}_{\text{new}} = \sum_{i=1}^n \alpha_i k(\mathbf{x}_i, \mathbf{x}_{\text{new}})$$

prediction for *many* new samples:

$$\hat{\mathbf{y}}_{\text{new}} = K_{\text{new}} \boldsymbol{\alpha}, \quad (K_{\text{new}})_{ij} = k(\mathbf{x}_{\text{new},i}, \mathbf{x}_j)$$

In the dual form all $\mathbf{x}$ appear as dot products ($\mathbf{x}_i^\top \mathbf{x}_j$ or $X X^\top$), thus we can replace them with the kernel function $k(\mathbf{x}_i, \mathbf{x}_j)$ and the kernel matrix K .

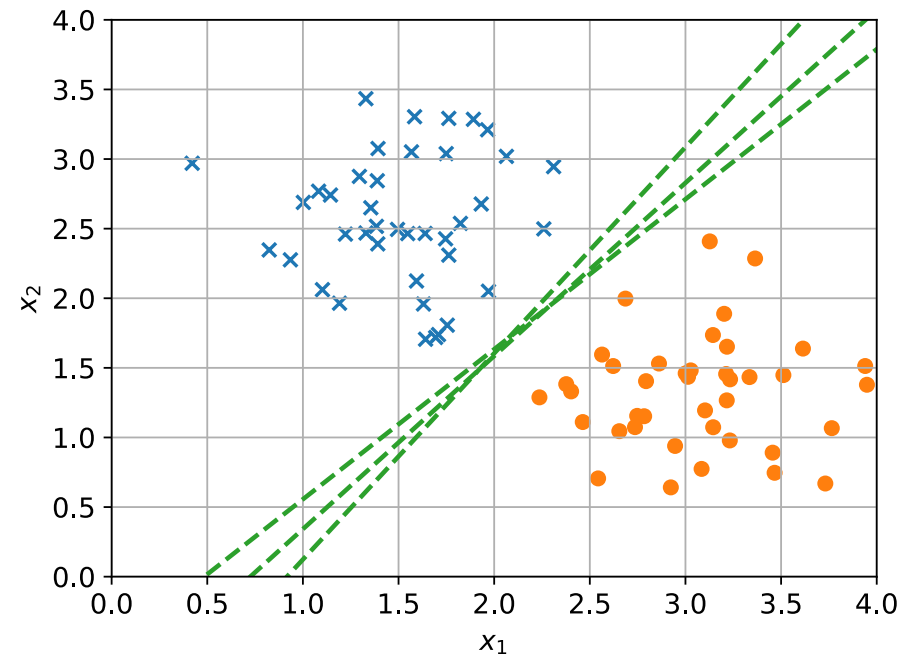
This implicit feature transformation gives the same results as the explicit one, but the computational complexity does not change with the dimensionality. [code example](#)

4 · Linear classification

Same line, different question: not how far, but on which side.

Linearly separable data

Definition: data are *linearly separable* if there exists a straight line (hyperplane in higher dimensions) that separates all positive from all negative samples without error.



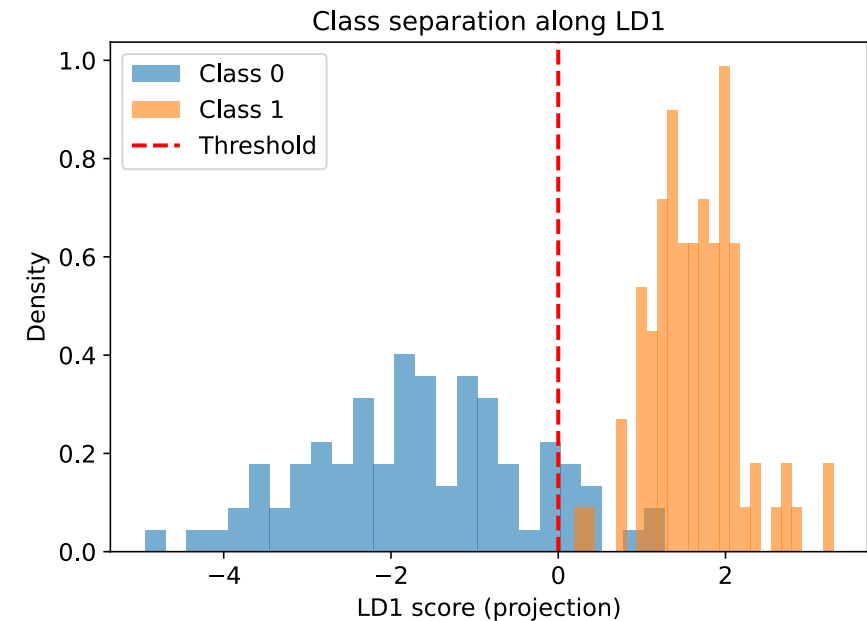
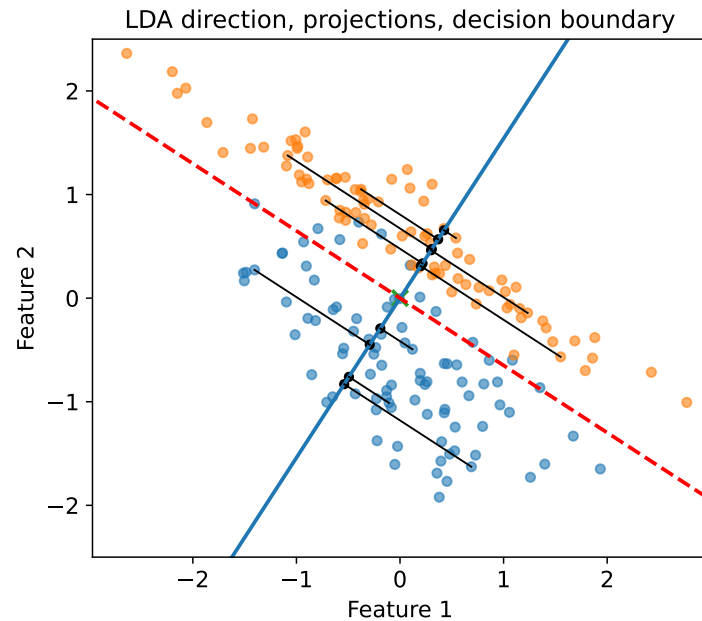
Real data are noisy, overlapping or mislabeled $\Rightarrow$ exact separability almost never holds. It is a theoretical concept needed for SVMs and for perceptron convergence.

Fisher criterion (maximize separation / penalize spread):

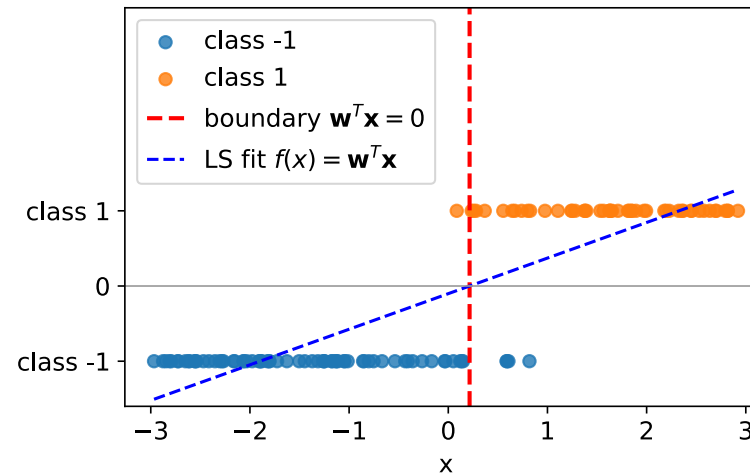
$$J(\mathbf{w}) = \frac{(\tilde{\mu}_1 - \tilde{\mu}_2)^2}{\tilde{s}_1^2 + \tilde{s}_2^2} = \frac{(\mathbf{w}^\top (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2))^2}{\mathbf{w}^\top S_W \mathbf{w}}, \quad S_W = S_1 + S_2.$$

$$\mathbf{w} = S_W^{-1} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$$

$$t^* = \frac{1}{2} (\boldsymbol{\mu}_1 + \boldsymbol{\mu}_2)^\top \mathbf{w} - \ln \frac{\pi_1}{\pi_2}$$



You can train with MSE (same as regression) even with labels $y \in \{-1, +1\}$ and then use $\text{sign}(\cdot)$ to get a label from the continuous prediction.



- simple, with closed-form solution
- sensitive to outliers
- bad performance

Idea: we should squeeze output values into the range $[-1, 1]$ (or $[0, 1]$), e.g. with a sigmoid.

Model (probabilistic):

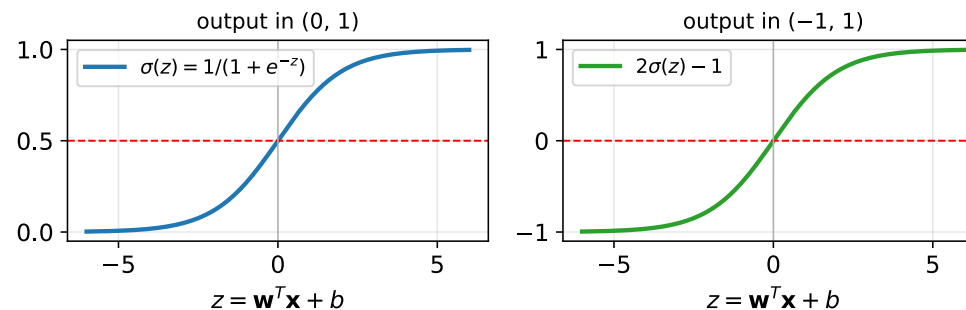
$$p(y = 1 \mid \mathbf{x}) = \sigma(z), \quad z = \mathbf{w}^\top \mathbf{x} + b, \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

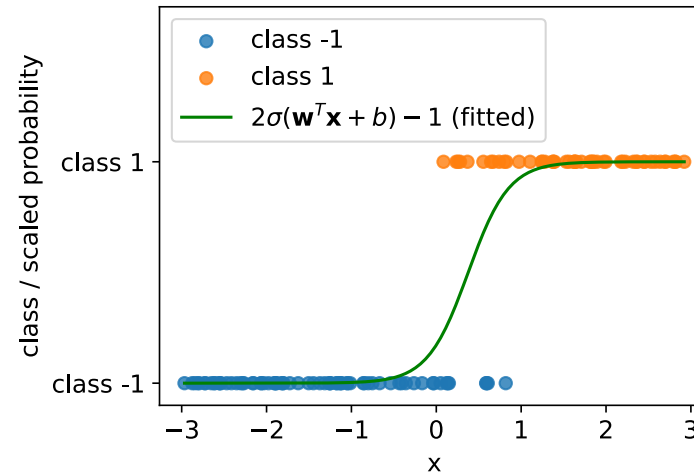
Decision rule:

$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(z) \geq 0.5 \\ 0 & \text{otherwise} \end{cases} \quad \text{equivalently} \quad \hat{y} = \text{sign}(z) \text{ for } y \in \{-1, +1\}.$$

Loss (cross-entropy / logistic):

$$\mathcal{L}_{\log}(y, z) = \begin{cases} -y \log \sigma(z) - (1 - y) \log(1 - \sigma(z)) & \text{for } y \in \{0, 1\}, \\ \log(1 + e^{-yz}) & \text{for } y \in \{-1, +1\}. \end{cases}$$



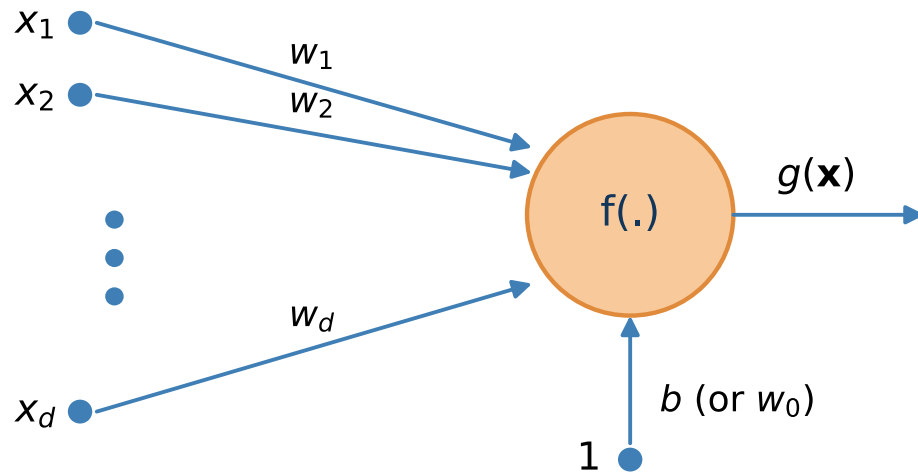


- Outputs calibrated probabilities (under the correct model).
- Convex objective; solution e.g. by gradient descent.
- Less sensitive to outliers than MSE.
- We can add regularization.
- This loss is used for deep learning (the linear model is replaced by a neural network).

More on this in the last lecture.

Perceptron algorithm

27



$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b), \quad y \in \{-1, +1\}.$$

$$\hat{y} = \text{sign}(\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}) = \text{sign} \left(\sum_{j=0}^d w_j x_j \right)$$

$$\mathcal{L}_{\text{perc}}(y, \tilde{\mathbf{w}}; \tilde{\mathbf{x}}) = \max(0, -y \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}})$$

Intuition: if a sample is misclassified ($y \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} \leq 0$), we *shift* $\tilde{\mathbf{w}}$ toward $y \tilde{\mathbf{x}}$. Multiplication by $y \in \{-1, +1\}$ automatically changes the sign depending on the sample label.

Simple iterative rule (learning rate $\eta > 0$):

$$\text{for } i = 1, \dots, n : \quad \text{if } y_i \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_i \leq 0, \quad \tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \eta y_i \tilde{\mathbf{x}}_i.$$

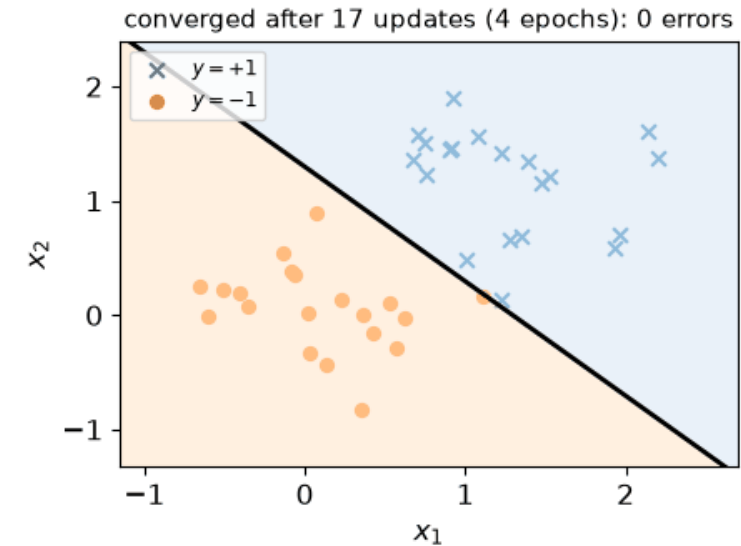
This algorithm provably converges only for linearly separable classes.

Perceptron algorithm: code

28

```
1 def train_perceptron(X, y, epochs):
2     n, d = X.shape
3     w = np.zeros(d)
4     for _ in range(epochs):
5         for i in range(n):
6             y_hat = X[i] @ w
7             if y[i] * y_hat <= 0:
8                 w += y[i] * X[i]
9     return w
```

```
1 def predict_perceptron(X, w):
2     return np.sign(X @ w)
```



Key idea: each update adds $y_i \mathbf{x}_i$ to $\mathbf{w} \Rightarrow$ the weights have to be a weighted sum (linear combination) of the samples: $\mathbf{w} = \sum_{i=1}^n \alpha_i \mathbf{x}_i$. That is the definition of the weights α in the dual form — an intuitive “proof” of the dual form. [runnable example](#)

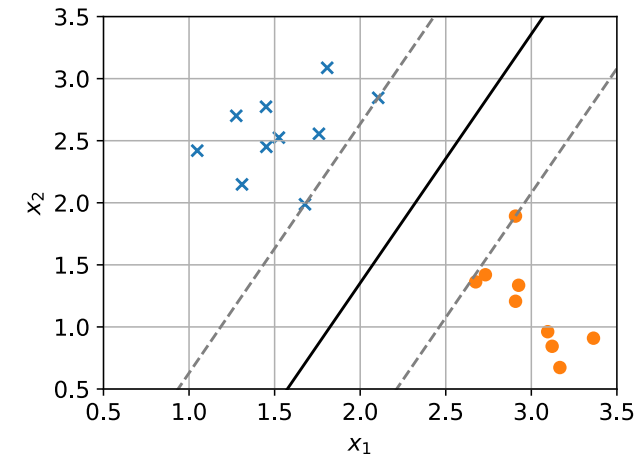
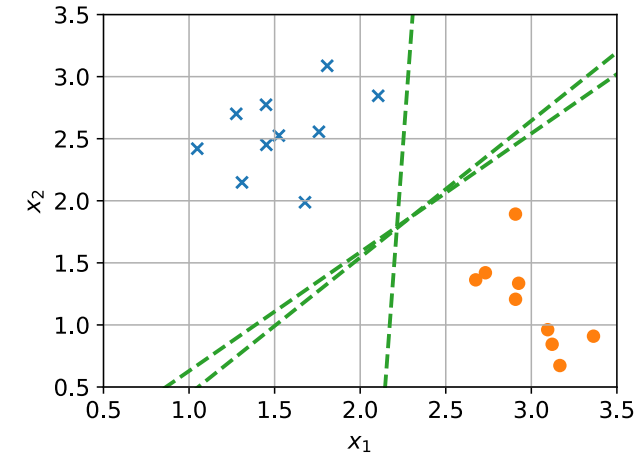
5 · Support vector machines

If many boundaries are correct, take the one with the widest safety zone.

For linearly separable data, which decision boundary should we choose if all of them give the best classification?

The best one can be defined as the one with the largest **margin** (“largest safe space”).

Optimization problem: choose the separating hyperplane that maximizes the margin, i.e. the distance between the decision boundary and the closest points from each class. How do we formulate this mathematically?



From intuition to SVM optimization

31

Step 1: margin definition. The distance of a point to the hyperplane $(\mathbf{w}, b)$ is

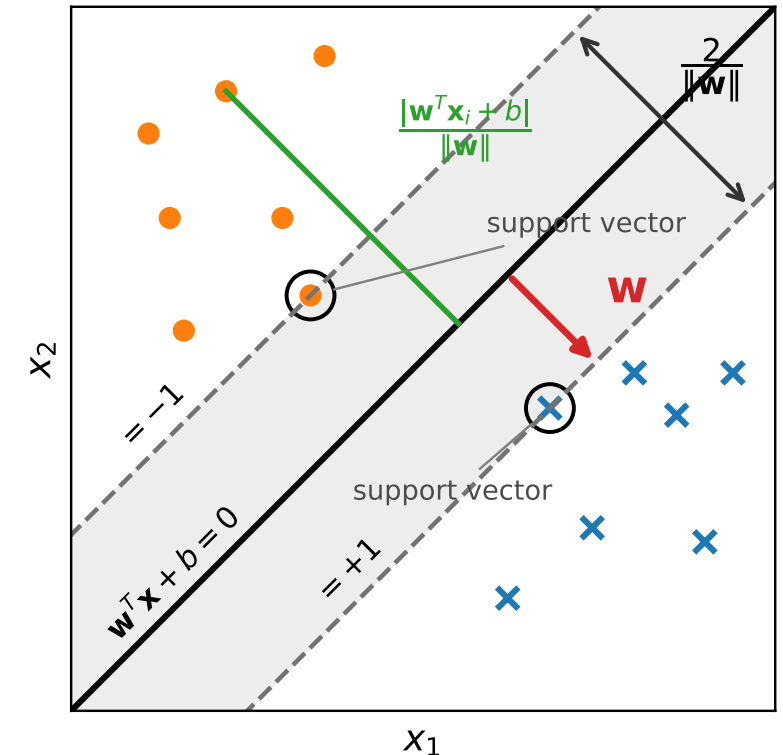
$$\frac{|\mathbf{w}^\top \mathbf{x}_i + b|}{\|\mathbf{w}\|}$$

Step 2: margin for correctly classified points. For all i : $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \Rightarrow$ the closest points (support vectors) lie exactly at distance $1/\|\mathbf{w}\|$.

Step 3: optimization. Maximize the margin $1/\|\mathbf{w}\|$

$$\underset{\mathbf{w}, b}{\text{maximize}} \quad \frac{1}{\|\mathbf{w}\|} \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$$

Equivalent standard form: $\underset{\mathbf{w}, b}{\text{minimize}} \quad \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1.$



Intuition 1: the ≥ 1 condition just fixes the scale — we force the closest points to sit exactly at distance 1, so the margin is measured as $1/\|\mathbf{w}\|$ and is well defined.

Intuition 2: instead of forcing $\|\mathbf{w}\| = 1$ and letting the margin vary, we fix the margin at 1 and let the scale of $\mathbf{w}$ adjust — so the margin becomes $1/\|\mathbf{w}\|$.

Optimization problem (primal):

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n$$

Geometric margin of the closest points: $1/\|\mathbf{w}\|$ (margin band width = $2/\|\mathbf{w}\|$).

Dual (convex QP):

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad \sum_{i=1}^n \alpha_i y_i = 0, \quad \alpha \geq 0$$

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i, \quad \hat{y} = \text{sign}\left(\sum_{i \in \text{SV}} \alpha_i y_i \mathbf{x}_i^\top \mathbf{x} + b\right).$$

It is a *quadratic programming* (QP) problem (in the dual) — we can use standard QP tools, i.e. efficient algorithms with guaranteed global optima.

Support vectors (SV):

$$\alpha_i > 0 \iff y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1 \quad (\text{they lie on the margin and determine } \mathbf{w}, b).$$

We need only the SVs for prediction in the dual ($\alpha_i y_i \mathbf{x}_i^\top \mathbf{x}$ is zero for $\alpha_i = 0$) $\rightarrow$ very fast prediction. Intuitively: only the SVs are necessary to get the same line. [code example](#)

Hard-margin SVM by hand: two points

33

$\mathbf{x}_1 = (1, 1), y_1 = +1$ and $\mathbf{x}_2 = (-1, -1), y_2 = -1$.

Geometry: the boundary is the perpendicular bisector $x_1 + x_2 = 0$; each point is at distance $\sqrt{2}$ from it, so the margin is $1/\|\mathbf{w}\| = \sqrt{2}$ and $\|\mathbf{w}\| = 1/\sqrt{2}$.

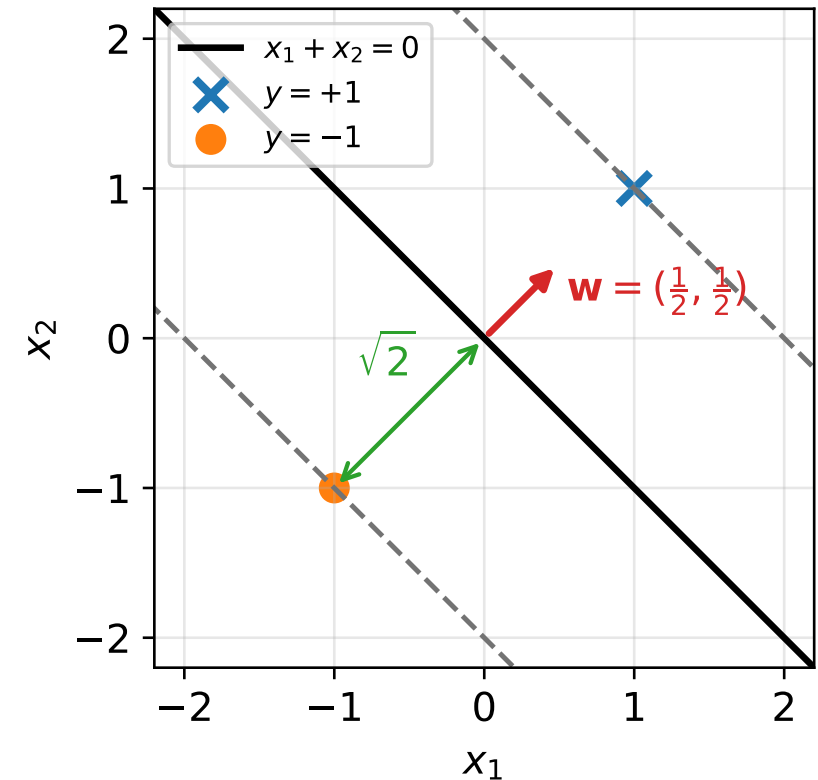
Primal: $\mathbf{w} = (\frac{1}{2}, \frac{1}{2}), b = 0$ — check $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$ for both points; objective $\frac{1}{2} \|\mathbf{w}\|^2 = \frac{1}{4}$.

Dual: $\mathbf{x}_1^\top \mathbf{x}_1 = \mathbf{x}_2^\top \mathbf{x}_2 = 2, \mathbf{x}_1^\top \mathbf{x}_2 = -2$; the constraint $\alpha_1 y_1 + \alpha_2 y_2 = 0$ gives $\alpha_1 = \alpha_2 = \alpha$:

$$\sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j = 2\alpha - \frac{1}{2} \cdot 2 (2\alpha)^2 = 2\alpha - 4\alpha^2 \Rightarrow \alpha = \frac{1}{4}$$

$$\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i = \frac{1}{4}(1, 1) + \frac{1}{4}(1, 1) = (\frac{1}{2}, \frac{1}{2}) \quad \checkmark$$

Dual optimum $2 \cdot \frac{1}{4} - 4 \cdot \frac{1}{16} = \frac{1}{4}$ = primal optimum (strong duality); both points are support vectors.



$$\boxed{\underset{\mathbf{w}, b}{\text{minimize}} \quad \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1.} \quad \text{works only if the data are linearly separable.}$$

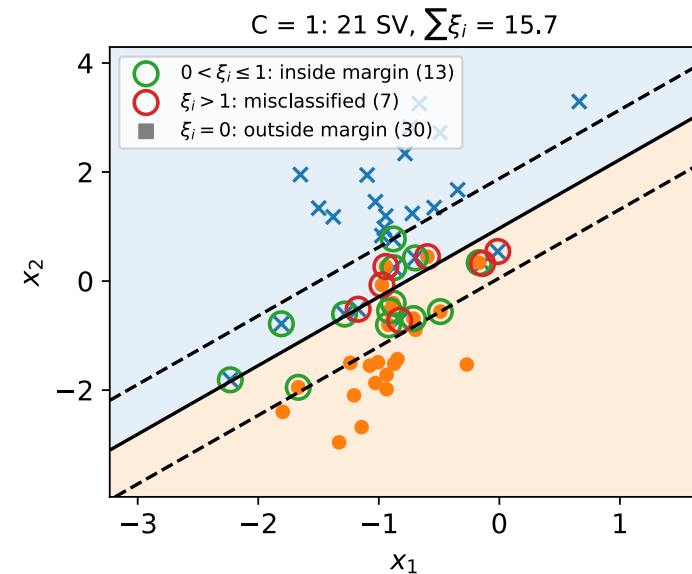
Allow violations with penalization via slack variables $\xi_i \geq 0$:

$$\underset{\mathbf{w}, b, \xi}{\min} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

$C > 0$ trades off margin size vs. violations:

- $\xi_i = 0$: correctly classified with margin ≥ 1 (30 points in the figure),
- $0 < \xi_i \leq 1$: correct but inside the margin (13),
- $\xi_i > 1$: misclassified (7).

Every point with $\xi_i > 0$ is a support vector (21 of 50 here); the slacks add up to $\sum_i \xi_i = 15.7$.



$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Eliminate ξ_i analytically: from the constraints, the smallest feasible

$$\xi_i = \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)).$$

Plugging back gives the *unconstrained* problem:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)).$$

Check on the previous figure: for a misclassified point with $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = -0.5$ the slack is $\xi_i = 1.5$; for a point inside the margin with score 0.6 it is 0.4; outside the margin it is 0.

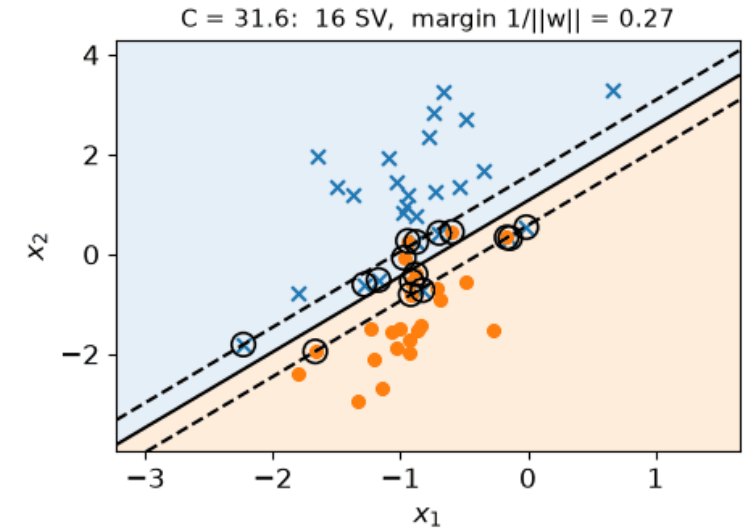
Soft-margin SVM (hinge loss form)

36

$$\min_{\mathbf{w}, b} \underbrace{\frac{1}{2} \|\mathbf{w}\|^2}_{\text{margin (regularization)}} + \underbrace{C \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b))}_{\text{hinge loss}}$$

Interpretation:

- $\frac{1}{2} \|\mathbf{w}\|^2$: prefers a large margin (smaller weights).
- Hinge loss: penalizes points *inside* the margin or misclassified.
- $C > 0$: trade-off — higher $C \Rightarrow$ fit the training data harder, lower $C \Rightarrow$ wider margin / more tolerance.
- Hinge loss can also be used for e.g. neural networks.
- Same data, C from 0.01 to 30: margin $2.54 \rightarrow 0.27$, support vectors $44 \rightarrow 16$.



C-SVM (standard soft-margin):

$$\min_{\mathbf{w}, b, \xi} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- Hyperparameter $C \in [0, \infty)$ controls the trade-off between margin width and training errors.
- Larger C : fewer violations, tighter fit, risk of overfitting.
- Smaller C : wider margin, more tolerance to errors, risk of underfitting.

ν -SVM (alternative parameterization):

$$\min_{\mathbf{w}, b, \xi, \rho} \quad \frac{1}{2} \|\mathbf{w}\|^2 - \nu \rho + \frac{1}{n} \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq \rho - \xi_i, \quad \xi_i \geq 0$$

- Uses the parameter $\nu \in (0, 1]$ instead of C .
- ν has an intuitive meaning:
 - upper bound on the fraction of margin errors (training errors),
 - lower bound on the fraction of support vectors.

Motivation: the soft-margin SVM in *dual form* depends only on dot products:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^{\top} \mathbf{x}_j \quad \text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C.$$

Kernel trick: replace the dot product with a kernel function

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j k(\mathbf{x}_i, \mathbf{x}_j) \quad \text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C.$$

Decision function:

$$f(\mathbf{x}) = \text{sign} \left(\sum_{i \in \text{SV}} \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b \right).$$

Summary: kernel SVM allows *nonlinear decision boundaries* in the input space while keeping the optimization problem the same size (it depends on n , not on the feature dimension).

Interpretation (linear SVM):

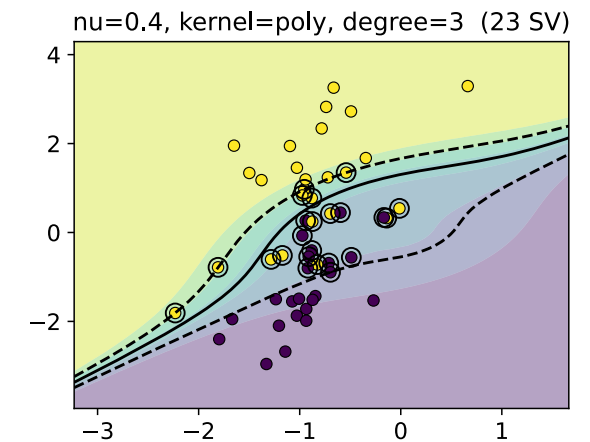
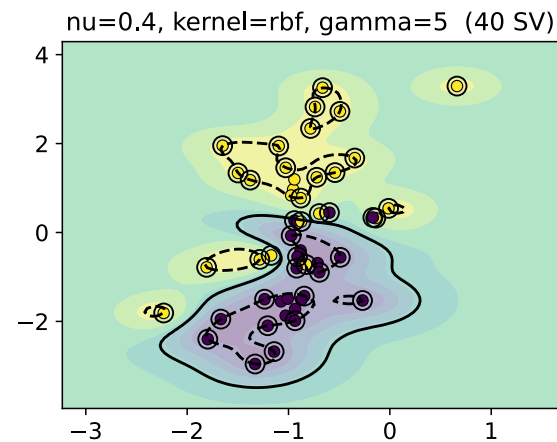
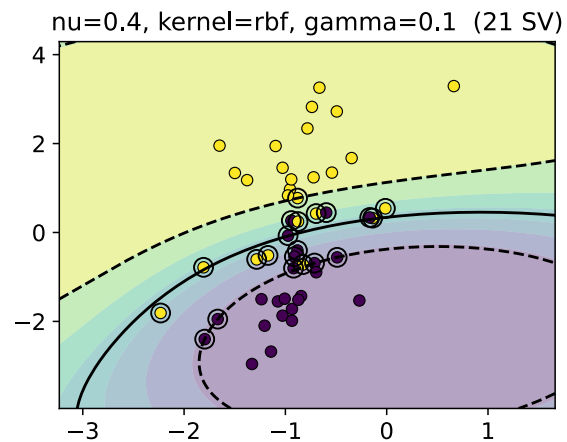
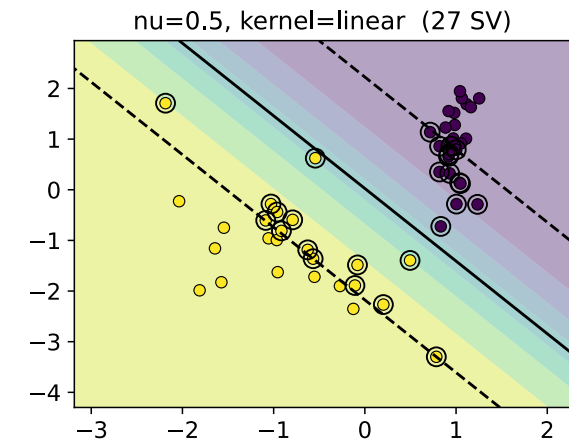
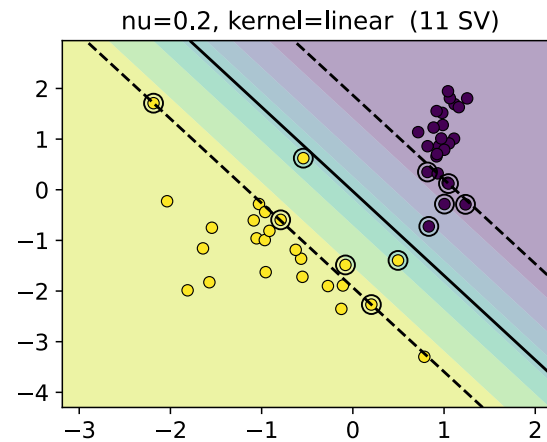
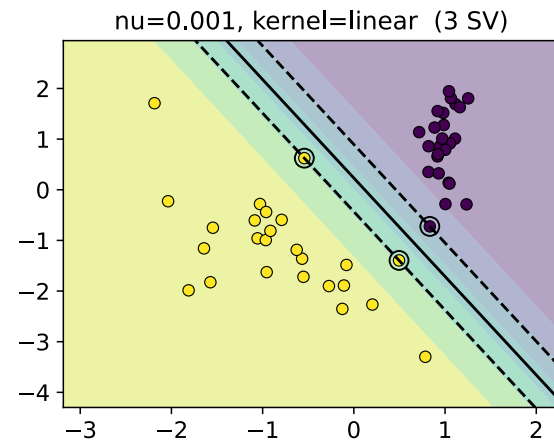
- $\mathbf{w}$: weights (importance) of features, $1/\|\mathbf{w}\|$ = margin (half of the band width $2/\|\mathbf{w}\|$).
- SVs: training points on or inside the margin; they alone define $\mathbf{w}$, b .

Interpretation (kernel SVM):

- No explicit $\mathbf{w}$ in the input space.
- α_i : dual coefficients (nonzero only for SVs). A larger α_i means more influence of that SV on the boundary.
 - Few SVs $\Rightarrow$ sparser, simpler model.
 - Many SVs $\Rightarrow$ more complex boundary, higher memory cost.

Hyperparameters to choose:

- C or ν — trade-off between margin width and training errors.
- Kernel type and kernel parameters:
 - Polynomial kernel: degree p — larger is a more complex model (over-fitting).
 - RBF (Gaussian): $\gamma = 1/(2\sigma^2)$ (inverse width) — larger is a more complex model.



[code example — visualization](#) · [code example — application](#)

6 · Recap

Three sets of questions to check that the whole chain from OLS to kernel SVM holds together.

Recap quiz I — linear models basics

42

- Explain the **bias trick**. How do we rewrite $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ using $\tilde{\mathbf{x}}$ and $\tilde{\mathbf{w}}$? Why is this practical?
- What is the difference between **regression** and **classification** for linear models? State the role of the decision boundary $\mathbf{w}^\top \mathbf{x} + b = 0$.
- Why is **MSE** sensitive to outliers while **MAE** is more robust? Give the intuition.
- Write the generic **empirical risk minimization** form: average loss + regularization. What are typical choices of $\Omega(\mathbf{w})$ and their effects?
- Compare **logistic regression** with the **MSE classifier**.
- In a linear classifier, what does a **larger margin** mean and how does it relate to $\|\mathbf{w}\|$?
- Give a scenario where you would prefer **MAE/LAD** over MSE. Briefly justify.

Recap quiz II — OLS, regularization, polynomials

43

- Derive the **closed-form solution for OLS** from the objective $g(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|_2^2$.
- Contrast **Ridge** (L_2) vs. **LASSO** (L_1): coefficient shrinkage, feature selection, and the existence of a closed form.
- How does the **hyperparameter** λ affect the bias–variance trade-off? Give examples of *underfitting* and *overfitting*.
- How do you build **polynomial regression** via feature expansion? Explain why it remains linear in the parameters.
- Using a simple example, explain how the **polynomial degree** and **regularization** jointly shape the fit.

- Explain how the **perceptron** update implies that $\mathbf{w}$ is a linear combination of the training samples: $\mathbf{w} = \sum_i \alpha_i \mathbf{x}_i$.
- Define the **margin** in SVM and explain why maximizing the margin leads to minimizing $\frac{1}{2} \|\mathbf{w}\|^2$.
- Contrast **hard-margin** vs. **soft-margin** SVM. What is the role of C and what do the **slack** variables ξ_i represent?
- Interpret **support vectors** and the dual coefficients α_i . What does *many* vs. *few* SVs mean for the model?
- Write the SVM **dual objective** and indicate where **dot products** (or kernels) appear between samples.
- Explain the **kernel trick**.
- Which **hyperparameters** do we tune in kernel SVMs (poly vs. RBF), and how do they affect the complexity of the decision boundary?
- Why is the dual/kernel approach beneficial when the feature dimension d is very large or infinite?